AI EXCEL ASSISTANT — COMPLETE ARCHITECTURE & CODE MANUAL

Full-Stack Natural Language Spreadsheet Analysis • Formula Engineering • Data Visualization • Cloud Ready

Author: Pavan Adithya | Target: Boss & Personal GitHub Repos | Theme: Vibrant Red & White

1. AI MODELS & API KEYS USED IN THIS PROJECT

LLM Tier	Provider	Exact Model Identifier	Python SDK	Env Variable	Operational Role in Application
PRIMARY	Google	gemini-2.5-flash gemini-1.5-flash	google-genai==2.31.0	GEMINI_API_KEY	Answers spreadsheet questions, validates schema columns, generates Excel formulas (=SUM, =AVERAGE, =XLOOKUP), recommends charts, explains analysis.
FALLBACK	OpenAI	gpt-4o-mini	openai==3.33.0	OPENAI_API_KEY	Automatic fallback service. Engages seamlessly if Gemini key is missing or quota/rate limits are hit. Guarantees zero downtime.
OFFLINE	Native	Smart Heuristic Rule Engine	Built-in (FastAPI + Pandas)	*None (Autonomous)*	Schema-grounded deterministic engine: computes direct Pandas previews, unique counts, distributions, summaries, and formula generation without API keys.

2. COMPLETE TECHNOLOGY STACK & EXACT VERSIONS

Category	Component	Version	Function & Practical Responsibility
Language	Python	3.11+	Underlying asynchronous execution runtime for backend processing and data calculations.
Web Framework	FastAPI	0.143.3	High performance asynchronous REST API with automatic OpenAPI Swagger docs (/docs).
ASGI Server	Uvicorn	0.54.0	Production ASGI web server handling HTTP requests and static asset streaming.
Spreadsheet Core	Pandas	3.0.6	High-speed DataFrame ingestion, aggregations, multi-column group-by, min/max/mean calculations.
Numerical Arrays	NumPy	2.4.6	Vectorized numeric calculations and smart column data type inference.
Excel File Engine	openpyxl	3.1.5	Direct read/write access to .xlsx files, sheet metadata, formatting, formula writing, chart embedding.
Data Plotting	Matplotlib	3.11.2	Core plotting engine creating bar, line, pie, and scatter charts saved as high-res PNG.
Plot Styling	Seaborn	0.13.2	Statistical palette and theme styling applied over Matplotlib graphs.
Database ORM	SQLAlchemy	2.1.5	Object-relational mapping models, query builder, and relationship links.
SQL Database	SQLite (ai_excel.db)	3.x (Built-in)	Persistent database storing users, uploaded files, chat logs, and analysis history.
Security & Hash	bcrypt	5.0.0	Direct password hashing with salt generation for tamper-proof credentials.
Tokens & JWT	python-jose	3.5.0	Signed HMAC-SHA256 JWT generation and validation for session authentication.
Email OTP Mailer	smtplib (Standard)	Built-in	Real-time TLS/SSL SMTP transmission of 6-digit verification codes to user mailboxes.
Testing Suite	pytest + httpx	9.1.1 / 0.28.1	Automated unit and integration test suite across auth, spreadsheets, chat, and history.
Frontend UI	HTML5 + CSS3 + JS	ES6 Vanilla	Vibrant Red & White Single Page App (SPA) featuring Chat, Uploads, Material Symbols Light.
Deployment 1	Render Cloud	Blueprint	PaaS hosting configured via render.yaml for automated web service build.
Deployment 2	Vercel Serverless	WSGI Edge	Edge serverless deployment configured via vercel.json + api/index.py.

3. SYSTEM APPLICATION & DATA PROCESSING FLOW

Step 1: Sign Up / Sign In Request (User provides credentials) • **Step 2: Real OTP Email Dispatch** (Server generates 6-digit code with 10-minute validity and delivers via TLS SMTP to user's real email) • **Step 3: User Verification** (User copies OTP from their email inbox, submits to frontend, server validates and issues signed JWT bearer token) • **Step 4: Spreadsheet Ingestion** (User uploads .xlsx/.xls/.csv ➔ Pandas extracts sheet metadata, detects domains, computes column stats ➔ saved to SQL) • **Step 5: Natural Language Query & Direct Dataframe Routing** (User asks question in chat or clicks suggested question ➔ routed to direct Pandas preview, unique counter, aggregation, formula builder, or Matplotlib visualizer) • **Step 6: Real-time UI Rendering & SQL History Persistence** (Formatted answer displayed with interactive table or chart, logged to SQLite database table analysis_history).

4. REAL EMAIL OTP & TWO-FACTOR AUTHENTICATION (2FA) DEEP-DIVE

To provide institutional-grade account security and prevent unauthorized access, the application enforces **Real Email Two-Factor Authentication (OTP)** on every user login and registration. The system connects directly to standard TLS/SSL SMTP mail servers (such as Google Gmail, Microsoft Outlook, or corporate mail gateways) to transmit a branded, tamper-resistant 6-digit confirmation code directly to the user's inbox.

Stage	Actor	Action & Technical Implementation Details
1. Credential Submission	User ➔ Client	User enters email and password into the direct sign-in form. Frontend transmits JSON payload to POST /api/auth/login.
2. Bcrypt Validation	FastAPI ➔ DB	Backend verifies password hash against database record using salted bcrypt. If invalid, returns 401 Unauthorized immediately.
3. OTP Generation & DB Lock	Auth Service	Server generates a cryptographically random 6-digit numeric OTP code (secrets.randbelow). Sets otp_code and otp_expires_at = UTC + 10 minutes in the User SQL table.
4. Real SMTP Transmission	Email Service	email_service.send_otp_email() builds a multipart (HTML + plain text) email with vibrant Red & White styling and connects to SMTP host (e.g. smtp.gmail.com:587 with STARTTLS) to deliver the code.
5. Inbox Retrieval	User Mailbox	User receives email from **Excel AI Assistant**, opens the message, and copies the prominent 6-digit verification code.
6. Code Verification & JWT Issuance	Client ➔ Server	User pastes the 6-digit code into the verification modal (POST /api/auth/verify-otp). Server checks expiration and match. Upon success, clears the OTP, marks account verified, and issues signed HMAC-SHA256 JWT.

HOW TO CONFIGURE REAL GMAIL / SMTP EMAIL DELIVERY IN .ENV

Configuring Google Gmail App Passwords (Recommended for Instant Setup):

1. Open Google Account Security (**https://myaccount.google.com/security**) ➔ Turn 2-Step Verification **ON**.
2. Search for **App Passwords** in Google Account search bar ➔ Name it **'Excel AI Assistant'**.
3. Google gives a 16-letter code (e.g. abcd efgh ijkl mnop). In .env (or Render/Vercel settings), enter:
   SMTP_HOST="smtp.gmail.com" | SMTP_PORT=587 | SMTP_USER="you@gmail.com" | SMTP_PASSWORD="your-16-char-code"
4. Save and restart server. Real verification emails will instantly arrive in your mailbox on every login!

5. COMPLETE CODEBASE MAP — EVERY FILE, LOCATION & RESPONSIBILITY

File Path / Location	Layer	Key Classes & Functions	Purpose & Implementation Details
app/main.py	Entry Point	app, lifespan(), mount()	Main FastAPI initialization, CORS with expose_headers, router registration, static files mounting.
app/config.py	Config	Settings, settings	Pydantic BaseSettings loading .env, database URLs, SMTP keys, LLM keys, and cloud flags.
app/database.py	Database	Base, engine, get_db()	SQLAlchemy engine, SessionLocal factory, declarative base, and request-scoped session injection.
app/models/*.py	ORM Models	User, ExcelFile, ChatSession, History	Relational database schema definitions mapping users, spreadsheets, chat threads, and analysis records.
app/services/email_service.py	Mailing	EmailService.send_otp_email()	Dispatches multipart HTML/text emails containing 6-digit codes via TLS/SSL SMTP (Gmail, Outlook, SES).
app/services/auth_service.py	Security	hash_password(), create_jwt()	Bcrypt salted hashing, 6-digit OTP generation, HMAC-SHA256 JWT token creation, and auth guards.
app/services/excel_service.py	Spreadsheet	inspect_excel(), domain_suggest()	Reads .xlsx/.xls/.csv, infers data types, detects domains (Project Mgmt, Finance), generates prompts.
app/services/chat_service.py	Orchestrate	process_user_query(), df_query()	Routes queries to direct Pandas computation (previews, unique counts, distributions) or LLM reasoning.
app/services/formula_service.py	Formulas	generate_formula(), validate()	Generates syntactically validated formulas (SUM, AVERAGE, XLOOKUP, IF) mapped strictly to real headers.
app/services/chart_service.py	Visuals	render_chart(), save_png()	Headless Matplotlib/Seaborn rendering of Bar, Line, Pie, and Scatter plots saved to static charts directory.
app/routers/*.py	REST API	auth, excel, chat, formula, history	FastAPI route controllers handling registration, login, OTP verification, file upload, chat, and history.
app/static/css/style.css	UI Styling	Red & White Studio Theme	Complete CSS stylesheet: crisp white canvas, radiant crimson accents, high contrast slate, smooth cards.
app/static/index.html	UI Template	Single Page App (SPA)	Direct auth screen, 3-column workspace, unique custom SVG emblem, Material Symbols Light icons.
app/static/js/app.js	UI Controller	Application Controller	Handles auth state, file uploads, chat stream rendering, interactive table previews, and document deletion.

6. CORE CODE LOGIC FROM SCRATCH — HOW THE ENGINE WORKS

6.1 Real Email Service with TLS SMTP (app/services/email_service.py)

```
import smtplib
from email.mime.multipart import MIMEMultipart
from email.mime.text import MIMEText
from app.config import settings

class EmailService:
    @staticmethod
    def send_otp_email(to_email: str, otp_code: str) -> bool:
        subject = f'Your Verification Code: {otp_code} — {settings.PROJECT_NAME}'
        body_html = f'''<div style="border:2px dashed #dc2626; padding:20px; text-align:center;">
                        <h2>Your 6-Digit OTP: <span style="color:#dc2626;">{otp_code}</span></h2>
                        <p>Valid for 10 minutes. Copy this code into the login prompt.</p></div>'''
        if settings.SMTP_HOST and settings.SMTP_USER and settings.SMTP_PASSWORD:
            msg = MIMEMultipart('alternative')
            msg['From'] = f'{settings.PROJECT_NAME} <{settings.FROM_EMAIL}>'
            msg['To'], msg['Subject'] = to_email, subject
            msg.attach(MIMEText(body_html, 'html'))
            with smtplib.SMTP(settings.SMTP_HOST, settings.SMTP_PORT) as server:
                server.starttls()
                server.login(settings.SMTP_USER, settings.SMTP_PASSWORD)
                server.send_message(msg)
            return True
        return False
```

6.2 Direct DataFrame Query & Preview Engine (app/services/chat_service.py)

```
def _handle_direct_dataframe_query(self, question: str, df: pd.DataFrame, file_record, session_id):
    q = question.lower().strip()
    # 1. Preview first N rows (Returns structured table for HTML rendering)
    if re.search(r'preview|first \d+ rows', q):
        preview_df = df.head(10).fillna('')
        return {'role': 'assistant', 'content': 'Here is a preview of the first 10 rows:',
                'analysis_result': {'is_table': True, 'columns': list(preview_df.columns),
                                    'rows': preview_df.to_dict(orient='records')}}
    # 2. Count unique entries for specific column
    match_uniq = re.search(r'how many unique (.+?) entries', q)
    if match_uniq:
        col = match_uniq.group(1).strip()
        return {'role': 'assistant', 'content': f'Found {df[col].nunique()} unique entries in {col}.'}
    # 3. Column Data Types Breakdown
    if 'data type' in q or 'data types' in q:
        types_rows = [{'Column Name': col, 'Data Type': str(df[col].dtype)} for col in df.columns]
        return {'role': 'assistant', 'content': 'Column Data Types Breakdown:',
                'analysis_result': {'is_table': True, 'columns': ['Column Name', 'Data Type'], 'rows': types_rows}}
```

6.3 Domain Detection & Dynamic Question Suggestions (app/services/excel_service.py)

```
def _infer_document_domain(columns: list) -> str:
    col_names = [c.lower() for c in columns]
    if any(k in col_names for k in ['task', 'assignee', 'progress', 'start date', 'deadline']):
        return 'Project Management & Operations'
    elif any(k in col_names for k in ['revenue', 'sales', 'profit', 'margin', 'price', 'cost']):
        return 'Financial & Sales Analytics'
    return 'General Tabular Dataset'

def generate_dynamic_suggestions(meta: dict) -> list:
    domain = meta.get('inferred_domain')
    # Generates domain-tailored questions without repeating generic dimensions
```

6.4 Headless Visualization Engine (app/services/chart_service.py)

```
import matplotlib
matplotlib.use('Agg') # Thread-safe non-GUI server backend
import matplotlib.pyplot as plt
import seaborn as sns

def render_chart(df: pd.DataFrame, chart_type: str, x_col: str, y_col: str, title: str, output_path: str):
    plt.figure(figsize=(8, 4.5))
    sns.set_theme(style='whitegrid', palette='Reds_r') # Vibrant Crimson Palette
    if chart_type == 'bar':
        sns.barplot(data=df.head(15), x=x_col, y=y_col, color='#dc2626')
        plt.xticks(rotation=40, ha='right')
    plt.title(title, fontsize=12, fontweight='bold', pad=12)
    plt.tight_layout()
    plt.savefig(output_path, dpi=160)
    plt.close()
```

7. REST API SPECIFICATION (FASTAPI SWAGGER / OPENAPI)

HTTP	Route Endpoint	Auth Required	Payload / Parameters	Response Description
POST	/api/auth/register	No	{email, full_name, password}	Registers user & dispatches real 6-digit OTP code to email.
POST	/api/auth/login	No	{email, password}	Verifies credentials & sends 6-digit 2FA login OTP to email.
POST	/api/auth/verify-otp	No	{email, otp}	Validates OTP, marks account verified, returns signed JWT token.
POST	/api/auth/resend-otp	No	{email}	Generates and dispatches a fresh 6-digit OTP to the email address.
GET	/api/auth/me	Bearer JWT	None	Returns current authenticated user profile and database usage stats.
POST	/api/excel/upload	Bearer JWT	multipart/form-data (.xlsx, .csv)	Parses sheet names, column headers, datatypes, row counts into SQL.
GET	/api/excel/files	Bearer JWT	None	Lists all user uploaded documents stored in the database.
DELETE	/api/excel/{file_id}	Bearer JWT	file_id in path	Permanently deletes uploaded spreadsheet from database & disk.
POST	/api/chat/ask	Bearer JWT	{file_id, question, session_id}	Executes natural language analysis: returns table preview, metric, or chart.
GET	/api/chat/sessions	Bearer JWT	None	Returns all user conversation threads from SQL database.
DELETE	/api/chat/sessions/{id}	Bearer JWT	id in path	Deletes conversation thread safely and idempotently without errors.
GET	/api/history	Bearer JWT	None	Returns chronological log of all formula & analysis records.
DELETE	/api/history/{id}	Bearer JWT	id in path	Deletes single analysis record from database.
GET	/health	No	None	Health check returning DB connectivity and configured LLM providers.

8. LOCAL SETUP & 1-CLICK LAUNCH INSTRUCTIONS

Step 1: Clone Repositories
git clone https://github.com/Pavanadithya12/excel-ai-assistant.git
Step 2: Automated 1-Click Launch (Windows)
Double-click start.bat (or run .\start.ps1). Automatically verifies Python 3.11, initializes virtual environment, installs dependencies, and launches Uvicorn at **http://localhost:8000**.
Step 3: Manual Startup
python -m venv .venv | .\.venv\Scripts\activate | pip install -r requirements.txt
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload

9. CLOUD DEPLOYMENT & AUTOMATED TESTING

Render Deployment (Blueprint PaaS):
1. In **dashboard.render.com** ➔ Click **New +** ➔ **Blueprint** ➔ Connect GitHub repository.
2. Render reads render.yaml, installs packages from UTF-8 requirements.txt, and launches Uvicorn on $PORT.
3. In Environment Variables, configure SMTP_USER, SMTP_PASSWORD, and GEMINI_API_KEY.
**Note on Free Tier: Render sleeps after 15m inactivity. Allow 50-60s on first request to wake up the service.*
Vercel Serverless Deployment:
Run vercel --prod or connect Git. Automatically builds via api/index.py with ephemeral SQLite routing.
Automated Test Suite:
Run pytest -v. Executes 15 comprehensive unit tests covering auth, bcrypt hashing, OTP lifecycle, Excel metadata extraction, formula validation, and Pandas calculations.